

Lab 1

Section 1: Practice Exercises

1) Use this dataset for all exercises:

```
pyspark -master local[*]

students = spark.createDataFrame([
    (101, "Ali", "CS", 88),
    (102, "Mona", "CS", 95),
    (103, "Youssef", "IT", 72),
    (104, "Nour", "IT", 81),
    (105, "Sara", "AI", 91),
    (106, "Omar", "AI", 85)
], ["student_id", "name", "major", "score"])

students.show()
```

```
>>> students = spark.createDataFrame([
...     (101, "Ali", "CS", 88),
...     (102, "Mona", "CS", 95),
...     (103, "Youssef", "IT", 72),
...     (104, "Nour", "IT", 81),
...     (105, "Sara", "AI", 91),
...     (106, "Omar", "AI", 85)
... ], ["student_id", "name", "major", "score"])

>>>
>>> students.show()
+-----+-----+-----+-----+
|student_id|  name|major|score|
+-----+-----+-----+-----+
|      101|   Ali|  CS|   88|
|      102|  Mona|  CS|   95|
|      103|Youssef| IT|   72|
|      104|  Nour| IT|   81|
|      105|  Sara| AI|   91|
|      106|  Omar| AI|   85|
+-----+-----+-----+-----+
```

a. Exercise 1: Find all students in the AI major with scores greater than 90.

```
students.filter((students.major == "AI") & (students.score > 90)).show()
```

```
>>> students.filter((students.major == "AI") & (students.score > 90)).show()
+-----+-----+-----+-----+
|student_id|name|major|score|
+-----+-----+-----+-----+
|      105|Sara|  AI|   91|
+-----+-----+-----+-----+
```

b. Exercise 2: Create a new column called result (Pass → score ≥ 80 - Fail → otherwise)

```
from pyspark.sql.functions import when, col
```

```
students.withColumn("result", when(col("score") >= 80, "Pass").otherwise("Fail")).show()
```

```
>>> from pyspark.sql.functions import when, col
>>> students.withColumn("result", when(col("score") >= 80, "Pass").otherwise("Fail")).show()
+-----+-----+-----+-----+-----+
|student_id|  name|major|score|result|
+-----+-----+-----+-----+-----+
|      101|   Ali|  CS|   88|  Pass|
|      102|  Mona|  CS|   95|  Pass|
|      103|Youssef| IT|   72|  Fail|
|      104|  Nour| IT|   81|  Pass|
|      105|  Sara| AI|   91|  Pass|
|      106|  Omar| AI|   85|  Pass|
+-----+-----+-----+-----+-----+
```

c. **Exercise 3:** Calculate the average score per major sorted from highest to lowest.

```
from pyspark.sql.functions import avg, col

# Group by major and calculate average score, then sort descending
avg_score_per_major = students.groupBy("major").agg(
    avg("score").alias("avg_score")
).orderBy(col("avg_score").desc())

avg_score_per_major.show()
```

```
>>> from pyspark.sql.functions import avg, col
>>>
>>> # Group by major and calculate average score, then sort descending
... avg_score_per_major = students.groupBy("major").agg(
...     avg("score").alias("avg_score")
... ).orderBy(col("avg_score").desc())
>>>
>>> avg_score_per_major.show()
+-----+
|major|avg_score|
+-----+
|  CS |    91.5 |
|  AI |    88.0 |
|  IT |    76.5 |
+-----+
```

d. **Exercise 4:** Count the number of distinct majors.

```
from pyspark.sql.functions import countDistinct

# Method 1: Using countDistinct

distinct_major_count = students.select(countDistinct("major").alias("distinct_majors"))

distinct_major_count.show()

# Method 2: Using distinct() and count()

distinct_count_alternative = students.select("major").distinct().count()

print(f"Number of distinct majors: {distinct_count_alternative}")
```

```
>>> from pyspark.sql.functions import countDistinct
>>>
>>> # Method 1: Using countDistinct
... distinct_major_count = students.select(countDistinct("major").alias("distinct_majors"))
>>> distinct_major_count.show()
+-----+
|distinct_majors|
+-----+
|                3|
+-----+

>>>
>>> # Method 2: Using distinct() and count()
... distinct_count_alternative = students.select("major").distinct().count()
>>> print(f"Number of distinct majors: {distinct_count_alternative}")
Number of distinct majors: 3
```

e. **Exercise 5:** Rename major to department and remove student_id.

```
from pyspark.sql.functions import col
# Rename column and drop student_id
students_renamed = students.withColumnRenamed("major", "department").drop("student_id")
students_renamed.show()
```

```
>>> from pyspark.sql.functions import col
>>>
>>> # Rename column and drop student_id
... students_renamed = students.withColumnRenamed("major", "department").drop("student_id")
>>> students_renamed.show()
+-----+-----+-----+
| name|department|score|
+-----+-----+-----+
| Ali|CS|88|
| Mona|CS|95|
| Youssef|IT|72|
| Nour|IT|81|
| Sara|AI|91|
| Omar|AI|85|
+-----+-----+-----+
```

2) **Section 2:** Practice Exercises - Use the same students DataFrame.

a. **Exercise 1:** Calculate the running total of scores ordered by student_id.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

# Define window specification for running total
window_spec = Window.orderBy("student_id").rowsBetween(Window.unboundedPreceding, Window.currentRow)

# Calculate running total
running_total = students.withColumn(
    "running_total",
    sum("score").over(window_spec)
)
running_total.show()
```

```
+-----+-----+-----+-----+-----+
| student_id| name|major|score|running_total|
+-----+-----+-----+-----+-----+
| 101| Ali|CS|88|88|
| 102| Mona|CS|95|183|
| 103| Youssef|IT|72|255|
| 104| Nour|IT|81|336|
| 105| Sara|AI|91|427|
| 106| Omar|AI|85|512|
+-----+-----+-----+-----+-----+
```

- b. **Exercise 2:** Create a column: (above_major_avg - below_major_avg) based on each student's major average score.

```
from pyspark.sql.functions import avg, round

# Define window for major average
major_window = Window.partitionBy("major")

# Calculate major average and difference
students_with_avg = students.withColumn(
    "major_avg",
    round(avg("score").over(major_window), 2)
).withColumn(
    "diff_from_major_avg",
    round(col("score") - col("major_avg"), 2)
)

students_with_avg.show()
```

student_id	name	major	score	major_avg	diff_from_major_avg
105	Sara	AI	91	88.0	3.0
106	Omar	AI	85	88.0	-3.0
103	Youssef	IT	72	76.5	-4.5
104	Nour	IT	81	76.5	4.5
101	Ali	CS	88	91.5	-3.5
102	Mona	CS	95	91.5	3.5

- c. **Exercise 3:** Find the top 2 students from each major using a window function.

```
from pyspark.sql.functions import row_number, rank, dense_rank

# Method 1: Using row_number() - gives unique ranking, no ties
window_spec = Window.partitionBy("major").orderBy(col("score").desc())

top_2_by_major = students.withColumn(
    "rank",
    row_number().over(window_spec)
).filter(col("rank") <= 2)

top_2_by_major.show()
```

student_id	name	major	score	rank
105	Sara	AI	91	1
106	Omar	AI	85	2
104	Nour	IT	81	1
103	Youssef	IT	72	2
102	Mona	CS	95	1
101	Ali	CS	88	2

3) Section 3: Practice Exercises - Use this dataset:

```
contacts = spark.createDataFrame([
    (" ali123@example.com "),
    ("sara_99@gmail.com"),
    ("mona.test@yahoo.com"),
    (" ahmed2025@outlook.com ")
], ["email"])
```

a. **Exercise 1:** Extract the username and email provider into separate columns.

```
from pyspark.sql.functions import trim, lower, split, col, regexp_extract
```

```
# Extract username (before @) and provider (after @)
contacts_extracted = contacts.withColumn(
    "email_clean", trim(lower(col("email")))
).withColumn(
    "username", split(col("email_clean"), "@").getItem(0)
).withColumn(
    "provider", split(col("email_clean"), "@").getItem(1)
)

contacts_extracted.select("username", "provider").show()
```

username	provider
ali123	example.com
sara_99	gmail.com
mona.test	yahoo.com
ahmed2025	outlook.com

b. **Exercise 2:** Create a column containing only the domain name (gmail, yahoo, outlook, etc.).

```
# Extract domain name (before the .com/.net/etc.)
contacts_with_domain = contacts_extracted.withColumn(
    "domain",
    split(col("provider"), "\\.").getItem(0)
)

contacts_with_domain.select("email", "domain").show()
```

email	domain
ali123@example....	example
sara_99@gmail.com	gmail
mona.test@yahoo.com	yahoo
ahmed2025@outloo...	outlook

c. **Exercise 3:** Clean the email column by: (Removing leading/trailing spaces - Converting to lowercase)

```
# Remove spaces and convert to lowercase
contacts_cleaned = contacts.withColumn(
    "email_cleaned",
    lower(trim(col("email")))
)

contacts_cleaned.select("email", "email_cleaned").show()
```

email	email_cleaned
ali123@example....	ali123@example.com
sara_99@gmail.com	sara_99@gmail.com
mona.test@yahoo.com	mona.test@yahoo.com
ahmed2025@outloo...	ahmed2025@outlook...

d. **Exercise 4:** Count how many emails belong to each provider.

```
# Clean first, then group by provider
provider_counts = contacts.withColumn(
    "email_clean", lower(trim(col("email")))
).withColumn(
    "provider", split(col("email_clean"), "@").getItem(1)
).groupBy("provider").count()

provider_counts.show()
```

provider	count
gmail.com	1
outlook.com	1
yahoo.com	1
example.com	1

e. **Exercise 5:** Replace all digits in usernames with an empty string.

```
from pyspark.sql.functions import regexp_replace, trim, lower, split, col

# Method 1: Complete pipeline
result = contacts.withColumn(
    "email_clean", lower(trim(col("email")))
).withColumn(
    "username", split(col("email_clean"), "@").getItem(0)
).withColumn(
    "username_no_digits",
    regexp_replace(col("username"), "[0-9]", "")
).withColumn(
    "provider", split(col("email_clean"), "@").getItem(1)
)

result.select("username", "username_no_digits", "provider").show()
```

username	username_no_digits	provider
ali123	ali	example.com
sara_99	sara_	gmail.com
mona.test	mona.test	yahoo.com
ahmed2025	ahmed	outlook.com

4) **Section 4:** Practice Exercises - Use this dataset:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    to_date, year, month, dayofmonth,
    quarter, datediff, last_day, add_months,
    col
)

# Create DataFrame
events = spark.createDataFrame([
    (1, "2025-01-15"),
    (2, "2025-03-22"),
    (3, "2025-07-04"),
    (4, "2025-10-19"),
    (5, "2025-12-31")
], ["event_id", "event_date"])
```

a. **Exercise 1:** Convert event_date into DateType.

```
events_date = events.withColumn(
    "event_date",
    to_date(col("event_date"), "yyyy-MM-dd")
)

events_date.show()
```

event_id	event_date
1	2025-01-15
2	2025-03-22
3	2025-07-04
4	2025-10-19
5	2025-12-31

b. **Exercise 2:** Extract (Year , Month, Day into separate columns.)

```
events_parts = events_date \
    .withColumn("year", year("event_date")) \
    .withColumn("month", month("event_date")) \
    .withColumn("day", dayofmonth("event_date"))

events_parts.show()
```

event_id	event_date	year	month	day
1	2025-01-15	2025	1	15
2	2025-03-22	2025	3	22
3	2025-07-04	2025	7	4
4	2025-10-19	2025	10	19
5	2025-12-31	2025	12	31

c. **Exercise 3:** Determine the quarter of each event.

```
events_quarter = events_date.withColumn(
    "quarter",
    quarter("event_date")
)

events_quarter.show()
```

event_id	event_date	quarter
1	2025-01-15	1
2	2025-03-22	1
3	2025-07-04	3
4	2025-10-19	4
5	2025-12-31	4

d. **Exercise 4:** Calculate how many days remain until the end of the event's year.

```
from pyspark.sql.functions import *

events_remaining = events_date.withColumn(
    "year_end",
    to_date(concat(year("event_date"), lit("-12-31")))
).withColumn(
    "days_remaining",
    datediff(col("year_end"), col("event_date"))
)

events_remaining.select(
    "event_id",
    "event_date",
    "days_remaining"
).show()
```

event_id	event_date	days_remaining
1	2025-01-15	350
2	2025-03-22	284
3	2025-07-04	180
4	2025-10-19	73
5	2025-12-31	0

e. **Exercise 5:** Find all events that occurred in the second half of the year (July–December).

```
second_half_events = events_date.filter(
    month("event_date") >= 7
)

second_half_events.show()
```

event_id	event_date
3	2025-07-04
4	2025-10-19
5	2025-12-31

5) Section 5: Practice Exercises - Use this dataset:

```
courses = spark.createDataFrame([
    ("Python", ["Programming", "Beginner", "Data"]),
    ("Spark", ["Big Data", "Programming"]),
    ("SQL", ["Database", "Data"]),
    ("Airflow", ["Workflow", "Automation", "Data"]),
    ("Linux", ["OS", "Administration"])
], ["course_name", "topics"])
```

a. **Exercise 1:** Find all courses that have more than 2 topics.

```
from pyspark.sql.functions import size, col
courses.filter(size(col("topics")) > 2).show(truncate=False)
```

course_name	topics
Python	[Programming, Beginner, Data]
Airflow	[Workflow, Automation, Data]

b. **Exercise 2:** Create a column has_data_topic (True/False).

```
from pyspark.sql.functions import array_contains, col
courses_with_flag = courses.withColumn(
    "has_data_topic",
    array_contains(col("topics"), "Data")
)
courses_with_flag.show(truncate=False)
```

course_name	topics	has_data_topic
Python	[Programming, Beginner, Data]	true
Spark	[Big Data, Programming]	false
SQL	[Database, Data]	true
Airflow	[Workflow, Automation, Data]	true
Linux	[OS, Administration]	false

c. **Exercise 3:** Use explode() and count how many courses contain each topic.

```
from pyspark.sql.functions import *
topic_counts = courses \
    .select(explode(col("topics")).alias("topic")) \
    .groupBy("topic") \
    .count()
topic_counts.show()
```

topic	count
Data	3
Programming	2
Administration	1
Big Data	1
Workflow	1
OS	1
Automation	1
Database	1
Beginner	1

d. **Exercise 4:** Find all courses that contain the topic "Programming".

```
from pyspark.sql.functions import array_contains, col
courses.filter(
    array_contains(col("topics"), "Programming")
).show(truncate=False)
```

course_name	topics
Python	[Programming, Beginner, Data]
Spark	[Big Data, Programming]

e. **Exercise 5:** Create a column containing the number of topics for each course.

```
from pyspark.sql.functions import size, col
courses_with_count = courses.withColumn(
    "num_topics",
    size(col("topics"))
)
courses_with_count.show(truncate=False)
```

course_name	topics	num_topics
Python	[Programming, Beginner, Data]	3
Spark	[Big Data, Programming]	2
SQL	[Database, Data]	2
Airflow	[Workflow, Automation, Data]	3
Linux	[OS, Administration]	2

Capstone Project — Online Learning Platform Analytics

Use the following datasets:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.window import Window
spark = SparkSession.builder.appName("Capstone Analytics").getOrCreate()

students = spark.createDataFrame([
    (1, "Ali"),
    (2, "Mona"),
    (3, "Sara"),
    (4, "Omar"),
    (5, "Nour")
], ["student_id", "student_name"])

enrollments = spark.createDataFrame([
    (1, "Python", 88),
    (1, "SQL", 92),
    (2, "Python", 95),
    (2, "Spark", 90),
    (3, "SQL", 78),
    (3, "Spark", 85),
    (4, "Python", 70),
    (5, "Spark", 98)
], ["student_id", "course", "score"])
```

1) Part 1: Data Preparation

1. Join students with enrollments.

```
from pyspark.sql import functions as F

joined = students.join(enrollments, "student_id", "inner")
joined.show()
```

student_id	student_name	course	score
5	Nour	Spark	98
1	Ali	Python	88
1	Ali	SQL	92
3	Sara	SQL	78
3	Sara	Spark	85
2	Mona	Python	95
2	Mona	Spark	90
4	Omar	Python	70

2. Check schema and data quality.

```
joined.printSchema()

print("Row count:", joined.count()) -> Row count: 8

print("Null counts:")

joined.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in joined.columns]).show()
```

```
>>> joined.printSchema()
root
 |-- student_id: long (nullable = true)
 |-- student_name: string (nullable = true)
 |-- course: string (nullable = true)
 |-- score: long (nullable = true)
```

student_id	student_name	course	score
0	0	0	0

3. Remove duplicate records if present.

```
print("Row count before dedup:", joined.count())

joined = joined.dropDuplicates()

print("Row count after dedup:", joined.count())

joined.show()
```

```
print("Row count before dedup:", joined.count())
Row count before dedup: 8
>>> joined = joined.dropDuplicates()
>>> print("Row count after dedup:", joined.count())
Row count after dedup: 8
>>> joined.show()

+-----+-----+-----+-----+
|student_id|student_name|course|score|
+-----+-----+-----+-----+
|5|Nour|Spark|98|
|1|Ali|Python|88|
|1|Ali|SQL|92|
|3|Sara|SQL|78|
|3|Sara|Spark|85|
|2|Mona|Python|95|
|2|Mona|Spark|90|
|4|Omar|Python|70|
+-----+-----+-----+-----+
```

2) Part 2: Aggregations

1. Calculate the average score per student.

```
avg_student = df.groupBy("student_id", "student_name") \
    .agg(avg("score").alias("avg_score"))
avg_student.show()
```

student_id	student_name	avg_score
5	Nour	98.0
1	Ali	90.0
3	Sara	81.5
2	Mona	92.5
4	Omar	70.0

2. Calculate the average score per course.

```
avg_course = df.groupBy("course") \
    .agg(avg("score").alias("avg_course_score"))
avg_course.show()
```

course	avg_course_score
SQL	85.0
Python	84.33333333333333
Spark	91.0

3. Find the highest score achieved in each course.

```
max_course = df.groupBy("course") \
    .agg(max("score").alias("max_score"))
max_course.show()
```

course	max_score
SQL	92
Python	95
Spark	98

4. Find the student with the highest average score overall.

```
top_student = avg_student.orderBy(desc("avg_score")).limit(1)
top_student.show()
```

student_id	student_name	avg_score
5	Nour	98.0

3) Part 3: Window Functions

1. Rank students within each course.

```
w_course = Window.partitionBy("course").orderBy(F.col("score").desc())
ranked = joined.withColumn("course_rank", F.rank().over(w_course))
ranked.show()
```

student_id	student_name	course	score	course_rank
1	Ali	SQL	92	1
3	Sara	SQL	78	2
2	Mona	Python	95	1
1	Ali	Python	88	2
4	Omar	Python	70	3
5	Nour	Spark	98	1
2	Mona	Spark	90	2
3	Sara	Spark	85	3

2. Calculate a running total of scores within each course.

```
w_running = Window.partitionBy("course").orderBy("score") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
running = ranked.withColumn("running_total", F.sum("score").over(w_running))
running.show()
```

student_id	student_name	course	score	course_rank	running_total
3	Sara	SQL	78	2	78
1	Ali	SQL	92	1	170
4	Omar	Python	70	3	70
1	Ali	Python	88	2	158
2	Mona	Python	95	1	253
3	Sara	Spark	85	3	85
2	Mona	Spark	90	2	175
5	Nour	Spark	98	1	273

3. Calculate the difference between a student's score and the course average.

```
w_avg = Window.partitionBy("course")
final = running.withColumn("course_avg", F.round(F.avg("score").over(w_avg), 2)) \
    .withColumn("diff_from_avg", F.round(F.col("score") - F.col("course_avg"), 2))

final.show(truncate=False)
```

student_id	student_name	course	score	course_rank	running_total	course_avg	diff_from_avg
3	Sara	SQL	78	2	78	85.0	-7.0
1	Ali	SQL	92	1	170	85.0	7.0
4	Omar	Python	70	3	70	84.33	-14.33
1	Ali	Python	88	2	158	84.33	3.67
2	Mona	Python	95	1	253	84.33	10.67
3	Sara	Spark	85	3	85	91.0	-6.0
2	Mona	Spark	90	2	175	91.0	-1.0
5	Nour	Spark	98	1	273	91.0	7.0

4) Part 4: Business Rules

1. Create a performance category:

Excellent → score ≥ 90
 Good → score between 80 and 89
 Needs Improvement → score < 80

```
df_perf = enrollments.withColumn(
    "performance_category",
    when(col("score") >= 90, "Excellent")
    .when((col("score") >= 80) & (col("score") < 90), "Good")
    .otherwise("Needs Improvement")
)
df_perf.show()
```

student_id	course	score	performance_category
1	Python	88	Good
1	SQL	92	Excellent
2	Python	95	Excellent
2	Spark	90	Excellent
3	SQL	78	Needs Improvement
3	Spark	85	Good
4	Python	70	Needs Improvement
5	Spark	98	Excellent

2. Count students in each performance category.

```
category_count = df_perf.groupBy("performance_category") \
    .count() \
    .orderBy(desc("count"))
category_count.show()
```

performance_category	count
Excellent	4
Good	2
Needs Improvement	2

5) Part 5: Final Report

1. Build a final DataFrame containing:

Student Name
 Course
 Score
 Course Rank
 Performance Category

2. Sort by:

Course

Course Rank

Display the final report.

```
w_final = Window.partitionBy("course").orderBy(F.col("score").desc())

final_report = (joined
    .withColumn("performance",
        F.when(F.col("score") >= 90, "Excellent")
        .when(F.col("score") >= 80, "Good")
        .otherwise("Needs Improvement"))
    .withColumn("course_rank", F.rank().over(w_final))
    .select("student_name", "course", "score", "course_rank", "performance")
    .orderBy("course", "course_rank")
)

final_report.show()
```

student_name	course	score	course_rank	performance
Mona	Python	95	1	Excellent
Ali	Python	88	2	Good
Omar	Python	70	3	Needs Improvement
Ali	SQL	92	1	Excellent
Sara	SQL	78	2	Needs Improvement
Nour	Spark	98	1	Excellent
Mona	Spark	90	2	Excellent
Sara	Spark	85	3	Good

Lab 2

1) Section 1: Practice Exercises -Use this dataset for all exercises:

```
employees = spark.createDataFrame([
    (1, "Anna", "Engineering", 95000),
    (2, "Ben", "Marketing", 72000),
    (3, "Clara", "Engineering", 88000),
    (4, "Dan", "HR", 65000),
    (5, "Eva", "Marketing", 79000),
    (6, "Frank", "Engineering", 91000),
], ["id", "name", "dept", "salary"])
```

1. Filter employees in the Engineering department with salary > 90,000.

```
q1 = employees.filter(
    (col("dept") == "Engineering") & (col("salary") > 90000)
)

q1.show()
```

id	name	dept	salary
1	Anna	Engineering	95000
6	Frank	Engineering	91000

2. Add a column salary_grade — "High" if salary $\geq$ 80,000, else "Standard".

```
q2 = employees.withColumn(
    "salary_grade",
    when(col("salary") >= 80000, "High")
    .otherwise("Standard")
)
q2.show()
```

id	name	dept	salary	salary_grade
1	Anna	Engineering	95000	High
2	Ben	Marketing	72000	Standard
3	Clara	Engineering	88000	High
4	Dan	HR	65000	Standard
5	Eva	Marketing	79000	Standard
6	Frank	Engineering	91000	High

3. Find the average salary per department, sorted descending.

```
q3 = employees.groupBy("dept") \
    .agg(avg("salary").alias("avg_salary")) \
    .orderBy(desc("avg_salary"))
q3.show()
```

dept	avg_salary
Engineering	91333.33333333333
Marketing	75500.0
HR	65000.0

4. Count distinct departments.

```
distinct_dept_count = employees.select("dept").distinct().count()
print("Distinct Departments =", distinct_dept_count) → Distinct Departments = 3
```

5. Rename dept to department and drop the id column.

```
q5 = employees.withColumnRenamed("dept", "department") \
    .drop("id")
q5.show()
```

name	department	salary
Anna	Engineering	95000
Ben	Marketing	72000
Clara	Engineering	88000
Dan	HR	65000
Eva	Marketing	79000
Frank	Engineering	91000

2) Section 2: Practice Exercises

- a. From the employees DataFrame, find the running total (cumulative sum) of salary ordered by id.

```
window_running = Window.orderBy("id") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

running_salary = employees.withColumn(
    "running_total_salary",
    sum("salary").over(window_running)
)
running_salary.show()
```

id	name	dept	salary	running_total_salary
1	Anna	Engineering	95000	95000
2	Ben	Marketing	72000	167000
3	Clara	Engineering	88000	255000
4	Dan	HR	65000	320000
5	Eva	Marketing	79000	399000
6	Frank	Engineering	91000	490000

- b. Tag each row with "above_avg" or "below_avg" based on whether the salary is above the group average for their department.

```
dept_avg = employees.groupBy("dept") \
    .agg(avg("salary").alias("dept_avg_salary"))
dept_avg.show()
```

dept	dept_avg_salary
Engineering	91333.33333333333
HR	65000.0
Marketing	75500.0

- c. Find the 3 employees with the highest salary in the Engineering department using a window function.

```
salary_tag = employees.join(dept_avg, "dept") \
    .withColumn(
        "salary_status",
        when(col("salary") > col("dept_avg_salary"),
            "above_avg")

        .otherwise("below_avg")
    )
salary_tag.show()
```

dept	id	name	salary	dept_avg_salary	salary_status
Engineering	1	Anna	95000	91333.33333333333	above_avg
Engineering	3	Clara	88000	91333.33333333333	below_avg
Engineering	6	Frank	91000	91333.33333333333	below_avg
HR	4	Dan	65000	65000.0	below_avg
Marketing	2	Ben	72000	75500.0	below_avg
Marketing	5	Eva	79000	75500.0	above_avg

3) Section 3: Practice Exercises

```
logs = spark.createDataFrame([
    ("2024-03-15 ERROR Connection timeout",),
    ("2024-03-16 INFO User logged in",),
    ("2024-03-17 WARN Disk space low",),
    ("2024-03-18 DEBUG Query executed",),
    ("2024-03-19 ERROR Database failure",)
], ["log_entry"])

logs = logs.withColumn("date", F.substring("log_entry", 1, 10)) \
    .withColumn("log_level", F.regexp_extract("log_entry", r"\d{4}-\d{2}-\d{2}\s+(\w+)", 1))
```

- a. From the log DataFrame, extract the date portion and the log level into separate columns.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

spark = SparkSession.builder.appName("StringFunctions").getOrCreate()

logs = spark.createDataFrame([
    ("2024-01-15 ERROR Something went wrong",),
    ("2024-06-03 INFO User logged in",),
    ("2024-12-25 WARN Disk space low",),
], ["log_message"])

# Extract date (first 10 chars) and log level (next word)
logs = logs.withColumn("date", substring(col("log_message"), 1, 10)) \
    .withColumn("log_level", trim(split(col("log_message"), " ").getItem(1)))

logs.show(truncate=False)
```

log_message	date	log_level
2024-01-15 ERROR Something went wrong	2024-01-15	ERROR
2024-06-03 INFO User logged in	2024-06-03	INFO
2024-12-25 WARN Disk space low	2024-12-25	WARN

b. Given a column of email addresses, extract the username (before @) and domain (after @).

```
emails = spark.createDataFrame([
    ("ali@example.com",),
    ("mona@gmail.com",),
    ("sara.company@outlook.co.uk",),
], ["email"])

emails = emails.withColumn("username", split(col("email"), "@").getItem(0)) \
    .withColumn("domain", split(col("email"), "@").getItem(1))

emails.show(truncate=False)
```

email	username	domain
ali@example.com	ali	example.com
mona@gmail.com	mona	gmail.com
sara.company@outlook.co.uk	sara.company	outlook.co.uk

c. Clean the name column — trim whitespace, convert to proper case, and replace any digits with empty string.

```
names = spark.createDataFrame([
    (" ali123 "),
    ("MONA456"),
    (" sara7_ahmed "),
    ("OMAR99"),
], ["name"])

cleaned = names.withColumn("cleaned_name",
    regexp_replace(
        initcap(trim(col("name"))), # trim + proper case
        r"\d",                     # any digit
        ""                          # replace with empty
    )
)

cleaned.show(truncate=False)
```

name	cleaned_name
ali123	Ali
MONA456	Mona
sara7_ahmed	Sara_ahmed
OMAR99	Omar

4) Section 4: Practice Exercises

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder.appName("DateExercises").getOrCreate()

# Create DataFrame
orders = spark.createDataFrame([
    (101, "Ali", "2026-01-15", 250),
    (102, "Mona", "2026-03-20", 500),
    (103, "Sara", "2026-06-03", 150),
    (104, "Omar", "2026-01-03", 800), # Saturday
    (105, "Nour", "2026-02-08", 300), # Sunday
    (106, "Ahmed", "2026-04-11", 450), # Saturday
    (107, "Laila", "2026-05-17", 600), # Sunday
    (108, "Khaled", "2026-06-06", 200), # Saturday
    (109, "Dina", "2026-06-07", 350), # Sunday
    (110, "Hassan", "2026-06-01", 900),
], ["order_id", "customer", "order_date", "amount"])

orders = orders.withColumn("order_date", to_date(col("order_date")))
```

a. From a list of orders with order_date strings, parse the dates and calculate:

- Age of the order in days
- Which quarter it belongs to (Q1-Q4)
- The last day of that order's month

a) Date calculations

today = current_date()

orders = orders \

.withColumn("age_days", datediff(today, col("order_date"))) \

.withColumn("quarter", concat(lit("Q"), quarter(col("order_date")))) \

.withColumn("last_day_of_month", last_day(col("order_date"))) \

.withColumn("day_name", date_format(col("order_date"), "EEEE"))

orders.show(truncate=False)

order_id	customer	order_date	amount	age_days	quarter	last_day_of_month	day_name
101	Ali	2026-01-15	250	139	Q1	2026-01-31	Thursday
102	Mona	2026-03-20	500	75	Q1	2026-03-31	Friday
103	Sara	2026-06-03	150	0	Q2	2026-06-30	Wednesday
104	Omar	2026-01-03	800	151	Q1	2026-01-31	Saturday
105	Nour	2026-02-08	300	115	Q1	2026-02-28	Sunday
106	Ahmed	2026-04-11	450	53	Q2	2026-04-30	Saturday
107	Laila	2026-05-17	600	17	Q2	2026-05-31	Sunday
108	Khaled	2026-06-06	200	-3	Q2	2026-06-30	Saturday
109	Dina	2026-06-07	350	-4	Q2	2026-06-30	Sunday
110	Hassan	2026-06-01	900	2	Q2	2026-06-30	Monday

b. Find all orders placed on a weekend (Saturday or Sunday).

b) Weekend orders (Saturday=6, Sunday=7 in Spark)

```
weekend_orders = orders.filter(dayofweek(col("order_date")).isin(1, 7))
```

```
weekend_orders.show(truncate=False)
```

order_id	customer	order_date	amount	age_days	quarter	last_day_of_month	day_name
104	Omar	2026-01-03	800	151	Q1	2026-01-31	Saturday
105	Nour	2026-02-08	300	115	Q1	2026-02-28	Sunday
106	Ahmed	2026-04-11	450	53	Q2	2026-04-30	Saturday
107	Laila	2026-05-17	600	17	Q2	2026-05-31	Sunday
108	Khaled	2026-06-06	200	-3	Q2	2026-06-30	Saturday
109	Dina	2026-06-07	350	-4	Q2	2026-06-30	Sunday

5) Section 5: Practice Exercises

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
spark = SparkSession.builder.appName("ArrayExercises").getOrCreate()

products = spark.createDataFrame([
    (1, "Wireless Mouse", ["electronics", "sale", "wireless"]),
    (2, "Gaming Keyboard", ["electronics", "gaming", "sale", "rgb"]),
    (3, "USB-C Hub", ["electronics", "accessories"]),
    (4, "Monitor Stand", ["furniture", "sale", "ergonomic", "office"]),
    (5, "Laptop Sleeve", ["accessories", "laptop"]),
    (6, "Webcam HD", ["electronics", "sale"]),
    (7, "Desk Lamp", ["furniture", "lighting", "sale", "modern"]),
], ["product_id", "product_name", "promo_tags"])
```

c. Find all products that have more than 2 tags.

```
more_than_2 = products.filter(size(col("promo_tags")) > 2)
```

```
more_than_2.show(truncate=False)
```

product_id	product_name	promo_tags
1	Wireless Mouse	[electronics, sale, wireless]
2	Gaming Keyboard	[electronics, gaming, sale, rgb]
4	Monitor Stand	[furniture, sale, ergonomic, office]
7	Desk Lamp	[furniture, lighting, sale, modern]

d. Create a new column has_sale_tag (True/False) from the promo_tags column.

```
products = products.withColumn(
    "has_sale_tag",
    array_contains(col("promo_tags"), "sale")
)
products.show(truncate=False)
```

product_id	product_name	promo_tags	has_sale_tag
1	Wireless Mouse	[electronics, sale, wireless]	true
2	Gaming Keyboard	[electronics, gaming, sale, rgb]	true
3	USB-C Hub	[electronics, accessories]	false
4	Monitor Stand	[furniture, sale, ergonomic, office]	true
5	Laptop Sleeve	[accessories, laptop]	false
6	Webcam HD	[electronics, sale]	true
7	Desk Lamp	[furniture, lighting, sale, modern]	true

e. Use `explode()` + `groupBy()` to count how many products each tag appears in.

```
tag_counts = products \  
  .select(explode(col("promo_tags")).alias("tag")) \  
  .groupBy("tag") \  
  .count() \  
  .orderBy(col("count").desc())  
tag_counts.show()
```

tag	count
sale	5
electronics	4
accessories	2
furniture	2
rgb	1
wireless	1
office	1
laptop	1
ergonomic	1
lighting	1
gaming	1
modern	1